Meta Production Engineer System Interview - Comprehensive Study Guide Summary

This guide consolidates information from three sources on Linux systems, troubleshooting, performance analysis, and production engineering concepts, with a focus on preparing for Meta Production Engineer interviews.

1. Meta PE Interview Overview

What is Production Engineering at Meta?

Production Engineering (PE) at Meta collaborates with engineering teams to ensure the Reliability, Scalability, Performance, and Security of production services. PEs are hybrid software and systems engineers, acting as a bridge between infrastructure and software, or teams and processes.

Interview Structure (Screening Round)

Two 45-minute interviews: 1. **Coding Interview**: Focuses on programming skills relevant to production engineering tasks. 2. **Production Engineering (PE) Basics**: Assesses system knowledge.

PE Basics Interview Format (45 minutes)

- **Structure**: 5 min introductions, 35 min technical assessment, 5 min Q&A/closing.
- **Assessment Areas**:
    - Embracing Ambiguity (Technical): Effectively working through unclear situations.
    - Operating Systems Concepts (PE): Theoretical understanding of system components, resources, and user interactions.
    - Web Technology (PE): Knowledge of internet/web processes and distributed systems theory.

Key Competencies Evaluated

Sold to
cxl0603@gmail.com

- Deep Linux Knowledge: Understanding kernel internals beyond surface-level commands.
- Troubleshooting Focus: A methodical approach to solving real production problems.
- Performance Analysis: Utilizing tools and metrics to find bottlenecks.
- System Design Thinking: Grasping how components interact at scale.

Preparation Strategy

- Focus on OS fundamentals: tooling, memory management, Unix process lifecycle.
- Use Linux as a primary desktop environment.
- Build and run simple websites on Linux systems.
- Work with HTTP client/server on Linux systems.

---

2. Linux Boot Process Deep Dive

Complete Boot Sequence Analysis

Phase 1: BIOS/UEFI Initialization

- **Process**: Power Button → Power-On Self Test (POST) → Hardware Detection → Boot Device Selection → MBR/GPT Loading.
- **POST**: Checks hardware integrity (memory, CPU, motherboard, screen, keyboard). BIOS software is on a ROM chip.
- **Boot Device Selection**: BIOS searches configured devices and loads the Master Boot Record (MBR) (first 512 bytes) from the selected bootable media.

Phase 2: Master Boot Record (MBR) & Boot Loader

- **Process**: MBR (512 bytes) → GRUB Stage 1 → GRUB Stage 2 → Kernel Selection.
- **MBR Structure**:
    - First 446 bytes: Primary boot loader code.
    - Next 64 bytes: Partition table (4 entries, 16 bytes each).
    - Last 2 bytes: Boot signature (0x55AA).
    - The MBR is located at the first sector of the hard disk (e.g., /dev/hda or /dev/sda).
- **GRUB (Grand Unified Bootloader)**:
    - Stage 1: Basic bootloader in MBR, limited by space.
    - Stage 1.5 (Optional): Intermediate stage for filesystem drivers.

Sold to
cxl0603@gmail.com

- Stage 2: Full GRUB with a menu interface, reads configuration from `/boot/grub/grub.cfg`. Resides under /boot. GRUB displays a splash screen for OS/kernel selection.
- Kernel Loading: Loads `vmlinuz` (compressed kernel image) and `initrd/initramfs` (initial RAM disk/filesystem) into RAM and passes control to the kernel.

Phase 3: Kernel Initialization

- **Process**: Kernel Decompression → Hardware Detection → initrd Mount → Driver Loading → Root Filesystem Mount.
- **Kernel Boot Steps**:

  1. Self-extraction: The compressed kernel decompresses itself.
  2. Hardware Detection: Enumerates CPU, memory, PCI devices. Initializes and configures computer memory and hardware.
  3. Memory Management: Sets up page tables and virtual memory.
  4. initrd/initramfs: Loads a temporary root filesystem into RAM. This contains programs needed to mount the real root filesystem.
  5. Essential Drivers: Loads storage, network, and filesystem drivers.
  6. Root Mount: Mounts the real root filesystem from storage. After the root file system is booted, initramfs is cleared from RAM.

- **initrd vs. initramfs**:

  - initrd: A RAM disk image with a traditional filesystem.
  - initramfs: A compressed cpio archive, generally more efficient.

Phase 4: Init System & Service Startup

- **Process**: PID 1 Start → Mount Filesystems → Start Services → User Login.
- **Init Process (PID 1)**:

  - The first userspace process executed by the kernel.
  - Parent of all other processes on the system.
  - Traditional systems use SysV init with runlevels. Its goal is to initialize the system based on the specified runlevel defined in `/etc/inittab`.
  - Modern systems use `systemd` with targets.
  - Provides services with different runlevels; responsible for starting, shutting down, and rebooting the OS.

Sold to
cxl0603@gmail.com

- **Systemd Boot Targets**:
  - `emergency.target` : Minimal system for recovery.
  - `rescue.target` : Single-user mode.
  - `multi-user.target` : Multi-user mode without a GUI.
  - `graphical.target` : Multi-user mode with a GUI.
- **Runlevels (Traditional)**:
  - 0: Halt system.
  - 1: Single-user mode (for maintenance).
  - 2: Multi-user mode without network services.
  - 3: Multi-user mode with network services (command line).
  - 4: User-definable.
  - 5: Multi-user mode with network services (GUI).
  - 6: Reboot system.
  - Runlevel programs are executed from `/etc/rc.d/rc*.d/` .
  - Check current runlevel: `who -r` . Set runlevel: `init {runlevel_number}` .
- **Filesystem Mounting**:
  - `/etc/fstab` defines mount points and options for filesystems.
  - `mount -a` mounts all filesystems listed in `/etc/fstab` (unless marked `noauto`).
  - Special filesystems like `/proc` , `/sys` , and `/dev` are also mounted.

Real Mode and Protected Mode

- **Real Mode**: Active during BIOS and early boot stages. Executes 16-bit instructions, uses 16-bit registers, and has a 1MB address space limit.
- **Protected Mode**: Entered after initial boot tasks. To transition, the Global Descriptor Table (GDT) is loaded into `gdtr` , and a bit in a control register (CR0) is set.

Boot Troubleshooting Scenarios

- **Scenario 1: System Hangs During Boot**
  - **Diagnostics**: Remove `quiet` and `splash` from GRUB kernel parameters; boot into single-user mode (`single` or `systemd.unit=rescue.target`) or emergency mode (`systemd.unit=emergency.target`); check logs (`journalctl -xb`).
  - **Common Causes**: Corrupted filesystem (use `fsck`), hardware failure (check

Sold to
cxl0603@gmail.com

`dmesg`), driver issues, configuration errors.

- **Scenario 2: Kernel Panic Analysis**
  - **Triggers**: Missing/corrupted initrd/initramfs, incorrect `root=` parameter in GRUB, hardware incompatibility/failure, corrupted kernel image.
  - **Recovery**: Boot from live USB/rescue media; mount root filesystem; chroot into the system; reinstall kernel; rebuild initramfs (`update-initramfs -u`); update GRUB (`update-grub`).
- **Scenario 3: GRUB Rescue Mode**
  - **Common Issues**: Corrupted GRUB configuration, missing kernel files, incorrect partition references.
  - **Manual Boot**: Use GRUB commands: `ls` (list partitions), `set root=(hdX,msdosY)` (set root partition), `linux /boot/vmlinuz... root=/dev/sdXY` (load kernel), `initrd /boot/initrd.img...` (load initrd), `boot`.

Boot Performance Optimization

- **Analysis**: `systemd-analyze` (total time), `systemd-analyze blame` (slowest services), `systemd-analyze critical-chain`, `systemd-analyze plot > boot.svg` (timeline visualization).
- **Optimizations**: Disable unnecessary services, optimize initramfs size, use SSD for boot partition, reduce GRUB timeout, enable parallel service startup.

---

3. Process & Memory Management Mastery

Process Architecture

Process Control Block (PCB) / Process Descriptor

Each process has a PCB containing: * PID (Process ID) and PPID (Parent Process ID). * Process State (e.g., Running, Sleeping, Zombie). * CPU registers, including Program Counter (address of the next instruction). * Memory limits and management info (e.g., page tables, segments). * List of open files/file descriptors. * Signal handlers and pending signals. * Scheduling information (priority, nice value). * Resource usage statistics. * Pointers to parent and child PCBs.

Sold to
cxl0603@gmail.com

Process States

- **Linux States**:
  - **R (Running/Runnable)**: Executing on CPU or waiting in the run queue.
  - **S (Interruptible Sleep)**: Waiting for an event to complete (e.g., I/O, signal), can be interrupted.
  - **D (Uninterruptible Sleep)**: Waiting for I/O, cannot be interrupted by signals. Often seen when a process waits for device I/O. Cannot be killed by `SIGKILL`; requires reboot or I/O response. Appears in load average.
  - **T (Stopped)**: Stopped by a job control signal (e.g., `SIGSTOP`) or because it's being traced (e.g., by a debugger).
  - **Z (Defunct/Zombie)**: Process has terminated, but its parent has not yet reaped it (called `wait()`). Zombies occupy space in the process table but don't use memory or CPU.
  - **X (Dead)**: Process being destroyed (less common to see).
- **States contributing to CPU load**: R (Running) and D (Uninterruptible Sleep).

Process Creation

1. **`fork()` System Call**:
   - Creates a new process by duplicating the calling (parent) process.
   - The child process gets:
     - An identical (but separate) copy of the parent's user-level virtual address space (code, data, heap, shared libraries, user stack).
     - Identical copies of the parent's open file descriptors, meaning the child can read/write any files open in the parent at the time of `fork()`. These FDs point to the same open file table entry.
     - A unique PID and its PPID set to the parent's PID.
     - Pending signals are not inherited.
   - `fork()` returns twice:
     - In the parent process: Returns the PID of the newly created child process.
     - In the child process: Returns 0.
     - Returns -1 on failure.
   - The parent resumes execution, and the child starts execution at the same point (where `fork()` returns).
   - Implemented by `clone()` system call, which calls `do_fork()`.

Sold to
cxl0603@gmail.com

    - `dup_task_struct()` : Creates new kernel stack, `thread_info` , and `task_struct` (initially identical).
    - Resource limit check: Ensures the new child doesn't exceed user's process limits.
    - Non-inherited members are cleared/initialized.
    - Child's state set to `TASK_UNINTERRUPTIBLE` initially.
    - `copy_flags()` : Updates flags (e.g., `PF_SUPERPRIV` cleared, `PF_FORKNOEXEC` set).
    - `alloc_pid()` : Assigns a new PID.
    - Child is woken up and run.

2. **Copy-on-Write (CoW)**:
    - A technique to optimize `fork()` .
    - Initially, parent and child share the same physical memory pages, marked as read-only.
    - If either process attempts to write to a shared page, a page fault occurs.
    - The kernel then creates a private copy of that page for the writing process, updating its page table.
    - This avoids copying data that might be shared or if the child immediately calls `exec()` .

3. **`exec()` Family of System Calls (e.g., `execve()`)**:
    - Replaces the current process image with a new program.
    - Creates a new address space and loads the new program into it. The OS stops the current process, loads the new program (e.g., `ls`), and starts it, replacing segments like stack, heap, and data.

4. **`vfork()` System Call**:
    - Similar to `fork()` , but with key differences:
        - Parent's page table entries are NOT copied; child shares parent's address space.
        - Parent process is blocked until the child calls `exec()` or `exit()` .
        - Child runs first.
        - Child is not allowed to write to the address space (though this isn't always strictly enforced, it can lead to corruption).
        - More efficient if the child immediately calls `exec()` , as it eliminates CoW overhead.

5. **`clone()` System Call**:
    - The underlying system call for `fork()` and `vfork()` , and also used to create threads.

Sold to
cxl0603@gmail.com

- Allows fine-grained control over what resources are shared between parent and child/thread.
- Example flags: `CLONE_VM` (share address space), `CLONE_FS` (share filesystem info), `CLONE_FILES` (share open files), `CLONE_SIGHAND` (share signal handlers).
- `fork()`: `clone(SIGCHLD, 0)`.
- `vfork()`: `clone(CLONE_VFORK | CLONE_VM | SIGCHLD, 0)`.
- Thread: `clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0)`.

6. **`wait()` / `waitpid()` / `wait4()` System Calls**:
   - Used by a parent process to wait for state changes in its children (e.g., termination).
   - Allows parent to retrieve child's exit status.
   - `wait4()` can return resource usage information of the terminated child, unlike `waitpid()`.
7. **`exit()` System Call**:
   - Terminates the calling process and frees its resources.
   - The process enters a zombie state until its parent calls `wait()` or `waitpid()`.

Zombie and Orphan Processes

- **Zombie Process**: A process that has completed execution (`exit()`) but still has an entry in the process table because its parent has not yet called `wait()` or `waitpid()` to read its exit status.
  - Zombies don't consume CPU or memory (apart from their process table entry).
  - Excessive zombies can fill the process table, preventing new processes from launching.
  - **Cleanup**: Signal the parent process (e.g., `kill -CHLD parent_pid`) or, if necessary, kill the parent process. `init` will then adopt and reap the zombie.
- **Orphan Process**: A process whose parent process terminates before the child does.
  - Orphaned processes are adopted by the `init` process (PID 1) or another process in the current thread group.
  - `init` periodically calls `wait()` to reap its adopted children, so orphans don't remain zombies indefinitely.

Init Process (PID 1)

- The first process started by the kernel during boot.

Sold to
cxl0603@gmail.com

- It is the ancestor of all other processes.
- Initializes the system based on runlevels or targets.
- Adopts orphan processes.

Inter-Process Communication (IPC)

Mechanisms for processes to communicate and synchronize: 1. **Pipes (Anonymous Pipes)**: * Unidirectional communication channel (byte stream). * Typically used between a parent and child process created via `fork()`. * Created using the `pipe()` system call, which returns two file descriptors (one for reading, one for writing). 2. **Named Pipes (FIFOs - First-In, First-Out)**: * Allows communication between unrelated processes. * Has a name and exists as a special file in the filesystem, created by `mkfifo`. * Can be bidirectional if two pipes are used. 3. **Shared Memory**: * Fastest IPC method as processes share a region of memory. * Requires explicit synchronization mechanisms (like semaphores or mutexes) to prevent race conditions. 4. **Message Queues**: * Allows processes to exchange messages via a kernel-managed queue. * Messages can be structured and may have priorities. * Supports producer-consumer patterns. 5. **Sockets**: * Provide full-duplex communication. * **Unix Domain Sockets**: Used for local IPC between processes on the same host. They use filesystem paths for addressing. * **Network Sockets (TCP/IP)**: Used for communication between processes on different hosts (or locally). 6. **Signals**: * Asynchronous notifications sent to a process or thread to indicate an event. * Used for process control (e.g., terminate, stop, continue) and basic communication. * Carry limited information (primarily the signal number itself). * Signals are produced by the kernel and stored in the PCB of the target process. * When a signal arrives, the process is interrupted, registers are saved, and it's trapped into kernel mode. The corresponding signal handler is invoked. Upon return, the interrupted activity may continue or terminate. * `SIGKILL` (9) and `SIGSTOP` (17/19) cannot be caught, blocked, or ignored. * Common signals: `SIGTERM` (15, graceful termination), `SIGINT` (2, Ctrl+C), `SIGHUP` (1, hangup/restart), `SIGCHLD` (child status change), `SIGPIPE` (write to broken pipe), `SIGUSR1` / `SIGUSR2` (user-defined). * To restart a process, `kill -HUP <PID>` can be used.

Nice Value & Process Priority

- **Nice Value**: A user-space value from -20 (highest priority, shortest timeslice) to +19 (lowest priority, longer effective timeslice for CPU-bound tasks but less frequent execution).
- Controls the proportion of CPU timeslice a process receives in traditional schedulers. In CFS, it influences vruntime progression.
- `nice -n <value> ./command` to start a command with a specific nice value.
- `renice -n <value> -p <PID>` to change the nice value of a running process.

Linux Scheduling

Sold to
cxl0603@gmail.com

- **Completely Fair Scheduler (CFS)**:
  - The default scheduler for normal (non-real-time) processes in Linux.
  - Core concept: **virtual runtime (vruntime)**. Each process has a vruntime, which is the amount of time it has spent running, weighted by its priority (nice value).
  - CFS always tries to run the process with the smallest (lowest) vruntime.
  - Uses a **red-black tree** to manage the list of runnable processes, ordered by vruntime. The leftmost node has the lowest vruntime.
  - Timeslices are dynamic, not fixed, based on the number of runnable processes and a target latency.
- **Real-Time Scheduling Policies**: For tasks requiring deterministic timing.
  - `SCHED_FIFO` (First-In, First-Out): Runs until it blocks, yields, or is preempted by a higher-priority real-time task. Lower-priority tasks cannot run if a `SCHED_FIFO` task is runnable.
  - `SCHED_RR` (Round-Robin): Similar to `SCHED_FIFO`, but each process runs for a fixed timeslice before being moved to the end of the queue for its priority level.
  - Trade-offs: Latency vs. Throughput.

Threads

- To the Linux kernel, there is no fundamental concept of a "thread" distinct from a "process". All threads are implemented as standard processes.
- A thread is essentially a process that shares certain resources (like address space, open files, signal handlers) with other processes (threads within the same parent process).
- **Creating Threads**: Uses `clone()` with flags specifying shared resources (e.g., `CLONE_VM`, `CLONE_FS`, `CLONE_FILES`, `CLONE_SIGHAND`). If address space is shared, writes by one thread are visible to others in the same process.
- **Kernel Threads**:
  - Special processes that run only in kernel space and do not have a user address space (their `mm` pointer is NULL).
  - They do not context switch into user-space.
  - Used by the kernel for background tasks (e.g., handling softirqs, flushing caches, `kswapd`).
  - Can only be created by another kernel thread.
  - All new kernel threads are forked from the `kthreadd` kernel process (PID 2).
  - `kthreadd` is spawned by the kernel during boot, not by `init`, so its PPID is 0.
  - Visible with `ps -ef`.

Sold to
cxl0603@gmail.com

Context Switching

- The process of saving the state of a currently running process (or thread) and restoring the state of another process (or thread) so that execution can be resumed on the latter.
- **Process**:

  1. Save the context (CPU registers, program counter, stack pointer, etc.) of the current process into its PCB.
  2. Move the PCB of the current process to the relevant queue (e.g., ready, blocked, I/O wait).
  3. The scheduler selects the next process to run from the ready queue.
  4. Load/restore the context of the new process from its PCB (registers, memory mappings, open files).
  5. Update page tables if necessary (may involve TLB flush).
  6. Pass control to the newly restored process to resume its execution.

- **Costs**: Register save/restore overhead, cache pollution (L1/L2 caches become cold for the new process), TLB flush costs (if address spaces change), memory barrier synchronization.
- **Monitoring**:

  - `vmstat 1` (see `cs` column for system-wide context switches per second).
  - `pidstat -w 1` (shows voluntary and involuntary context switches per process).
  - `perf record -e sched:sched_switch -g` (for detailed reasons).

- **System Call and Context Switch**: A system call does not necessarily lead to a context switch. It involves a mode switch (user to kernel). A context switch might occur if the syscall causes the process to block (e.g., waiting for I/O or a mutex that's unavailable).

Memory Management

Memory Management Unit (MMU)

- A hardware component that translates virtual memory addresses to physical memory addresses.
- It uses information stored in data structures like page tables (set up by the kernel) to perform this translation.
- The CPU always operates on virtual addresses, which are then translated by the MMU.
- Also enforces memory protection (access control) and handles cache control.
- Advantages: Efficient main memory use, prevents memory fragmentation, simplifies memory management, provides security by isolating address spaces.

Sold to
cxl0603@gmail.com

Virtual Address Space (VAS) Layout (Typical Linux)

A process's VAS is typically organized into segments: * **Kernel Space**: (Highest addresses, e.g., top 1GB on 32-bit, varies on 64-bit). Inaccessible to user-mode code. * **Stack**: Grows downwards from high user-space addresses. Used for local variables, function arguments, return addresses. * **(Gap)**: A region between stack and heap to detect collision. * **Memory Mapped Segment (mmap regions)**: For shared libraries, memory-mapped files, anonymous mappings. * **Heap**: Grows upwards from lower addresses. Used for dynamic memory allocation (e.g., `malloc`, `new`) managed by `brk()` / `sbrk()`. * **BSS (Block Started by Symbol)**: Uninitialized global and static variables. Zero-filled by the OS when the program loads. * **Data Segment**: Initialized global and static variables. * **Text Segment (Code Segment)**: Executable machine instructions. Usually read-only. (Lowest user-space addresses, e.g., starting around `0x08048000` on 32-bit).

Memory Allocation Mechanisms

- **`brk()` and `sbrk()`**: System calls used to change the program's data segment size (the "program break"), effectively growing or shrinking the heap.
- **`mmap()`**: Creates a new mapping in the virtual address space of the calling process. Can map files or devices into memory, or create anonymous mappings (not backed by a file, e.g., for heap-like allocation by `malloc` libraries).
- **`munmap()`**: Removes a mapping.

Page-Based Memory Management

- Modern MMUs divide the virtual address space into fixed-size **pages** (commonly 4KB on x86). Physical memory is also divided into **page frames**.
- The lower bits of a virtual address form the **offset** within a page (and are not translated).
- The upper bits form the **virtual page number (VPN)**.
- **Page Tables**: Hierarchical data structures (e.g., 4-level on x86_64) used by the MMU to map VPNs to **physical frame numbers (PFNs)**.
- **Translation Lookaside Buffer (TLB)**: An on-CPU cache for recently used page table entries (virtual-to-physical address translations) to speed up access.
- **Address Translation Process**:

  1. CPU generates a virtual address.
  2. MMU checks the TLB for a matching VPN.
  3. **TLB Hit**: If found, PFN is retrieved, physical address formed, memory accessed.
  4. **TLB Miss**: MMU (or OS with software-managed TLB) performs a page table walk to find the mapping.

     - If mapping found and page is in memory, TLB is updated, physical address formed.

Sold to
cxl0603@gmail.com

        - If mapping indicates page is not in memory (or invalid), a **page fault** occurs.

- **Page Fault**: An exception raised by the MMU when a process tries to access a page that is:
    - Not currently in physical memory (e.g., swapped out, or never loaded).
    - Mapped but with incorrect permissions (e.g., writing to a read-only page).
    - Not mapped at all (invalid address).
    - **Types**:
        - **Minor Fault**: Page is in memory, but MMU/TLB didn't know (e.g., needs to be added to process's page table). No disk I/O needed.
        - **Major Fault**: Page must be loaded from disk (e.g., from swap space or executable file). Involves disk I/O, so it's slow.
        - **Segmentation Fault / Protection Fault**: Access violates memory protection rules (e.g., writing to read-only text segment, accessing unmapped memory). Usually results in process termination (`SIGSEGV`).
    - Kernel page allocation strategy is different from user space (lazy allocation). Kernel allocates all pages at initialization. Memory violations in the kernel can lead to a system oops (kernel error). Page faults in kernel mode can cause a kernel oops. Therefore, kernel memory itself is generally not pageable (cannot be swapped to disk), though the data it manages (like user pages) can be.

Kernel Memory Zones

The kernel divides physical memory into zones to manage pages with different properties: * **`ZONE_DMA`** : Contains pages that can be used for Direct Memory Access (DMA) by older ISA devices (limited addressing capabilities, typically <16MB). Some hardware (like disk drivers) can perform DMA to specific memory addresses. DMA allows hardware to access memory independently of the CPU. * **`ZONE_DMA32`**: For DMA by 32-bit devices on 64-bit systems that can only address up to 4GB. * **`ZONE_NORMAL`**: Contains normal, regularly mapped kernel address pages. Permanently mapped into kernel space. * **`ZONE_HIGHMEM`**: (Primarily on 32-bit systems with >1GB RAM). Contains memory pages not permanently mapped into the kernel's address space. Requires special handling to access.

Slab Allocator / Layer

- A memory management mechanism within the kernel for allocating and freeing kernel objects (data structures) efficiently.
- Functions like a specialized cache or free list for frequently used kernel objects.
- **Structure**:

Sold to
cxl0603@gmail.com

- **Caches**: Divides different objects into groups called caches. There's one cache per object type (e.g., a cache for PCBs, a cache for inodes).
  - **Slabs**: Each cache consists of one or more contiguous blocks of physically contiguous memory called slabs.
  - **Objects**: Each slab contains multiple pre-initialized objects of the specific type.
  - Slab statuses: Full, Partial, Empty.

- **Process**:

  1. When the kernel needs a new instance of a data structure, it requests it from the slab allocator.
  2. The allocator tries to provide an already allocated but unused object from a partial slab (or an empty slab if no partial slabs exist).
  3. If no suitable empty or partial slab exists, a new slab is allocated from contiguous physical pages and added to the cache.
  4. When an object is freed, it's marked as unused within its slab.
  5. The free interface is called only when available memory grows low and the system is attempting to free memory.

- **Benefits**: Reduces memory fragmentation, speeds up allocation/deallocation by using pre-initialized objects, improves cache utilization.

Swap Space Management

- **Swap Space**: A portion of a hard disk (or a file) used as an extension to physical RAM when the system runs out of available physical memory.
- **Why Linux needs swap**:

  - When physical RAM is full and the system needs more memory, the kernel swaps out less frequently used pages of memory to swap space, freeing RAM for current applications.
  - Pages used by processes only during startup/initialization and then never used again can be swapped out, freeing memory for other processes.

- **Forms of Swap Space**:

  - **Swap Partition**: An independent disk partition dedicated solely for swapping; no other files can reside there.
  - **Swap File**: A special file within the regular filesystem that resides among other system and data files.

- **Advantages**:

Sold to
cxl0603@gmail.com

- Provides additional "virtual" memory when physical RAM is exhausted.
- Can move rarely needed items away from high-speed RAM.
- Allows hibernation (saving system state to disk).

- **Disadvantages**:
  - Disk I/O is significantly slower than RAM I/O, so accessing swapped-out pages is time-consuming and degrades performance.
  - Swap partitions take up fixed disk space and don't resize dynamically.
  - Can increase wear on HDDs/SSDs.

- **Swap Process (`kswapd`)**:
  1. Kernel detects memory pressure.
  2. `kswapd` daemon activates.
  3. Uses an LRU-like (Least Recently Used) algorithm to select inactive pages for eviction.
  4. **Anonymous pages** (e.g., process heap, stack, not backed by a file) are written to swap space.
  5. **File-backed pages** (part of the page cache):
     - If **clean** (not modified since read from disk), they can be discarded (can be re-read from the original file if needed).
     - If **dirty** (modified), they must be written back to their backing file on disk before the page frame can be reclaimed.

- **Swappiness**: A kernel parameter (`vm.swappiness`, 0-100, default 60) that controls how aggressively the kernel swaps memory pages versus dropping page cache pages.
  - Lower value (e.g., 10): Prefers dropping page cache over swapping anonymous memory. Better for systems where I/O performance for file data is critical.
  - Higher value (e.g., 60): Standard behavior.
  - 0: Disables swapping of anonymous pages unless absolutely necessary (OOM condition).

- **Thrashing**: A state where the system spends an excessive amount of time paging (swapping pages in and out) because memory is critically low. The CPU is busy swapping, making little progress on actual work.
- **Commands**:
  - View swap: `swapon -s`, `cat /proc/swaps`, `free -h`.
  - Create swap file: `dd if=/dev/zero of=/swapfile bs=1M count=1024`, `chmod 600 /swapfile`, `mkswap /swapfile`, `swapon /swapfile`. Add to

Sold to
cxl0603@gmail.com

`/etc/fstab` for persistence.
  - Set swappiness: `echo <value> > /proc/sys/vm/swappiness` or via `sysctl`.

Out of Memory (OOM) Killer

- A kernel mechanism invoked when the system is critically low on memory and cannot allocate more, even after trying to free pages.
- It occurs because all available memory, including swap space, has been allocated.
- The system becomes unable to load new programs, and existing programs may fail if they need more memory.
- **OOM Killer**: Selects one or more processes to kill to free up memory and prevent a complete system lockup.
  - It chooses which process(es) to kill based on an `oom_score` calculated for each process. The `oom_score` is viewable in `/proc/<PID>/oom_score`.
  - Factors influencing `oom_score`: memory usage, CPU time, privileges (root processes are less likely to be killed), `oom_score_adj` value.
  - `oom_score_adj` (in `/proc/<PID>/oom_score_adj`): Allows administrators to influence the likelihood of a process being killed (values from -1000 to +1000). A value of -1000 disables OOM killing for that process.
- **Dealing with High Memory Usage / OOM**:
  - High memory usage can force other processes to use swap, decreasing efficiency.
  - **Temporary Solutions**:
    - Increase the `nice` value of the high-memory process to decrease its priority if its speed is not critical, potentially reducing its memory request rate.
    - Stop unessential processes (`kill -SIGSTOP <PID>`) so the kernel can swap their memory to disk.
  - **Permanent Solutions**:
    - Restrict process memory usage with `ulimit` or `cgroups`, or by changing `limits.conf`.
    - Find and fix memory leaks using tools like `valgrind`, `ltrace`. If a process's memory usage grows persistently (seen via `top`), it might have a leak. High inactive memory (`vmstat`) can also indicate leaks.
    - Change process behavior or migrate it to another server.
    - Increase physical memory or optimize application memory usage.
  - **Monitoring**:

Sold to
cxl0603@gmail.com

- `free` or `vmstat` for system memory usage.
- `top` or `htop` for per-process memory usage.
- Check OOM Killer logs: `dmesg | egrep -i 'killed process'`.
- Disable panic on OOM: Change `/proc/sys/vm/panic_on_oom` from 1/2 to 0.
- Leave scripts to report memory usage from `/proc/*/status` for later analysis if issues are intermittent.

Memory Hierarchy

A tiered system of memory with different speeds and capacities: 1. CPU Registers (< 1ns access). 2. L1 Cache (1-2ns, small, per core). 3. L2 Cache (3-10ns, larger, per core or shared by a few cores). 4. L3 Cache (10-20ns, largest on-CPU cache, shared by all cores). 5. Main Memory (RAM) (50-100ns, GBs). 6. SSD Storage (0.1-1ms, TBs). 7. HDD Storage (5-10ms, TBs). When reading/writing, the processor first checks the cache. If data is present (cache hit), it accesses the cache instead of slower main memory. CPU cache entries typically contain a tag (memory address), data block, and flag bits (e.g., valid bit). Cache strategies include write-through/write-back and LRU for eviction.

Memory Performance Tuning & Analysis

- **Key Parameters (`sysctl vm.*` or `/proc/sys/vm/`)**:
  - `vm.swappiness`.
  - `vm.dirty_ratio`, `vm.dirty_background_ratio`: Control when dirty pages are written to disk.
  - `vm.overcommit_memory`: Policy for memory overcommitment (0=heuristic, 1=always allow, 2=never overcommit beyond swap + ratio of RAM).
  - `vm.overcommit_ratio`: Percentage of RAM to consider when `overcommit_memory` is 2.
  - `vm.min_free_kbytes`: Forces kernel to keep a minimum amount of memory free.
  - `vm.zone_reclaim_mode`: Controls aggressiveness of reclaiming memory from zones when they are full.
- **Memory Pressure Indicators**: High si/so in `vmstat`, many D-state processes, OOM killer activity, high page fault rates, decreasing available memory despite high cache.
- **Tools for Advanced Memory Analysis**:
  - `pmap -x <PID>` or `cat /proc/<PID>/maps` / `smaps`: Show process memory mappings.
  - `cat /proc/meminfo`: Comprehensive system memory stats.
  - `cat /proc/vmstat`: Detailed virtual memory statistics.

Sold to
cxl0603@gmail.com

- `cat /proc/buddyinfo` : Shows memory fragmentation per zone (buddy system allocator info).
- `cat /proc/slabinfo` : Kernel slab allocator statistics.
- `cat /proc/pagetypeinfo` : Page type statistics.
- Check logs for `kswapd` or `oom` activity: `grep -E "kswapd|oom" /var/log/messages` or `journalctl`.
- Check for allocation failures: `dmesg | grep -i "allocation failure"`.

- **Huge Pages**: Larger page sizes (e.g., 2MB or 1GB instead of 4KB) that can improve performance for applications managing large amounts of memory by reducing TLB misses and page table overhead.

  - Configure via `/proc/sys/vm/nr_hugepages`. Mount `hugetlbfs`. Applications use `mmap` with `MAP_HUGETLB`. Check support with `cat /proc/meminfo | grep Huge`.

---

4. Disk I/O & Filesystem Architecture

Virtual File System (VFS)

- An abstraction layer in the Linux kernel that provides a unified interface for user applications to access various types of filesystems (e.g., ext4, XFS, NFS).
- Allows commands like `open()`, `read()`, `write()`, `close()` to work consistently regardless of the underlying filesystem or storage device.
- **VFS Object Model**:

  1. **Superblock Object**: Represents a mounted filesystem. Contains metadata like filesystem type, block size, mount point info, total blocks/inodes, and pointers to filesystem-specific operations. One per mounted filesystem.
  2. **Inode Object**: Represents a file or directory (any filesystem object). Contains metadata about the object:

     - File mode (type and permissions: `i_mode`).
     - Owner UID (`i_uid`), GID (`i_gid`).
     - File size (`i_size`).
     - Timestamps (access `i_atime`, modification `i_mtime`, metadata change `i_ctime`).
     - Number of hard links to the file.
     - Pointers to data blocks or extents.

Sold to
cxl0603@gmail.com

    - Inode objects are constructed in memory only when files are accessed. Each superblock maintains a list of its inodes.

3. **Dentry (Directory Entry) Object**: Represents a directory entry, linking a name to an inode. Used for path name lookups and directory-specific operations.

    - Translates file path components into inodes.
    - **Dentry Cache (dcache)**: Caches dentry objects to speed up lookups. Contains all dentries. Organized as a hash table and LRU lists.
    - Dentry States:
        - **Used**: Corresponds to a valid inode, has active users. Linked with their associated inodes.
        - **Unused**: Corresponds to a valid inode, no current users. Kept in an LRU list.
        - **Negative**: Not associated with a valid inode (e.g., path doesn't exist or inode was deleted). Helps speed up lookups for non-existent files. Kept in an LRU list.

4. **File Object**: Represents an open file by a process. Created by `open()` and destroyed by `close()`.

    - Contains current file position/offset (can be changed by `lseek()`).
    - Access mode (read, write, append) with which the file was opened.
    - Reference to the associated dentry and inode.

- **Relationship**: File Object → Dentry Object → Inode Object → Data Blocks.
- **Workflow**: User-space `write()` → VFS `sys_write()` → Filesystem's specific write method → Physical device.

Filesystem Implementation Details (e.g., ext4)

- **Structure**:

    - **Superblock**: Global filesystem metadata.
    - **Block Groups**: Filesystem divided into manageable chunks. Each group has its own inode table, block bitmap, inode bitmap.
    - **Inode Table**: Stores inodes for files and directories within a block group.
    - **Block Bitmap**: Tracks free/used data blocks in a block group.
    - **Inode Bitmap**: Tracks free/used inodes in a block group.
    - **Data Blocks**: Store actual file content.
    - **Journal**: For metadata journaling (or full data journaling) to ensure filesystem consistency after crashes.

- **Key ext4 Features**: Extents (instead of indirect blocks), delayed allocation, multiblock

Sold to
cxl0603@gmail.com

allocation, journal checksumming, online defragmentation.

Block Devices & I/O Stack

- **Block Device**: A device that moves data in fixed-size blocks (e.g., hard drives, SSDs).
- **Terms**:
  - **Sector**: Smallest addressable unit on a block device (e.g., 512 bytes or 4KB).
  - **Block**: Kernel performs all disk operations in terms of blocks. Block size must be a multiple of sector size, a power of two, and no larger than page size.
- **Buffers (Buffer Cache)**: When a block is read or pending a write, it's stored in a buffer in memory. This is historically distinct from page cache but now largely unified. Buffers primarily hold metadata of files/data residing in the page cache. They contain info about directories, permissions, and track block device I/O.
- **Complete I/O Stack (Write Example)**: Application (`write()`) → System Call Interface (`sys_write()`) → VFS (`vfs_write()`) → Filesystem Layer (e.g., `ext4_file_write()`) → Page Cache (marks pages dirty) → Block Layer (`submit_bio()`) → I/O Scheduler → Device Driver → Storage Hardware.
- **Request Queues**: Pending I/O requests are stored here. High-level kernel code adds requests. If the queue isn't empty, the block driver takes requests and submits them to the device.

I/O Schedulers

- Manage a block device's request queue to optimize I/O performance, primarily by reducing disk seeks (for HDDs) and ensuring fairness.
- **Mechanisms**:
  - **Merging**: Combines adjacent requests into a single request.
  - **Sorting**: Reorders requests to minimize head movement (for HDDs).
- **Available Schedulers**:
  1. **NOOP (No Operation)**: Simple FIFO queue. Minimal CPU overhead. Best for devices that don't require reordering (e.g., SSDs, virtualized devices where host handles scheduling).
  2. **Deadline**: Uses separate read and write queues. Imposes a deadline for each request to prevent starvation. Good for database workloads.
  3. **CFQ (Completely Fair Queuing)**: Default for spinning disks in older kernels. Allocates timeslices to processes for I/O. Good for desktop/mixed workloads.

Sold to
cxl0603@gmail.com

4. **BFQ (Budget Fair Queuing)**: Aims for low latency and fairness. Good for interactive workloads and soft real-time applications. Often default on modern systems for HDDs.

- **Management**:
  - Check current scheduler: `cat /sys/block/sda/queue/scheduler`.
  - Change scheduler: `echo deadline > /sys/block/sda/queue/scheduler`.
  - Set at boot via kernel parameter: `elevator=deadline`.

Page Cache

- A memory cache used by the kernel to store recently accessed file data (pages of files) to minimize disk I/O. When a process issues `read()` / `write()`:
  1. Kernel checks if data is in page cache.
  2. **Cache Hit**: Data is read/written directly in RAM.
  3. **Cache Miss**: Kernel reads/writes data from/to disk and stores a copy in the page cache.
- **Cache Writing Policies**:
  - **Write-through**: Writes update both page cache and disk simultaneously. Simpler, safer, but slower.
  - **Write-back**: Writes update only the page cache in memory. The corresponding page is marked "dirty" and added to a dirty list. Dirty pages are written back to disk periodically (writeback) or when `sync()` / `fsync()` is called, or memory is low. Faster, but risk of data loss on crash if not synced.
- **Cache Eviction (Two-List Strategy / LRU variant)**:
  - Linux maintains an **active list** (hot pages, not readily available for eviction) and an **inactive list** (colder pages, candidates for eviction).
  - Pages move from inactive to active when accessed again.
  - Both lists are managed with an LRU-like strategy.
- **Flusher Threads (`pdflush` / `kworker`)**: Kernel threads responsible for writing dirty pages back to disk.
  - Triggered when: free memory shrinks below a threshold, dirty data ages past a threshold, or `sync()` / `fsync()` system calls are invoked.
- **Buffer Cache vs. Page Cache**:
  - Historically, buffer cache stored metadata (filesystem structures like superblocks, inodes,

Sold to
cxl0603@gmail.com

directory blocks) and raw block device data, while page cache stored file contents.
    - In modern Linux kernels, these are largely unified. "Buffers" in `free` command output often refer to metadata cached for block devices, while "cached" refers to the page cache for file contents.

- **Managing Cache**:
    - View stats: `cat /proc/meminfo` (Cached, Dirty, Writeback).
    - Force sync: `sync` (syncs all dirty data and metadata), `fsync(fd)` (syncs a specific file).
    - Drop caches (for testing only!): `echo 1 > /proc/sys/vm/drop_caches` (page cache), `echo 2 > /proc/sys/vm/drop_caches` (dentries and inodes), `echo 3 > /proc/sys/vm/drop_caches` (all).

Soft Links and Hard Links

- **Hard Link**:
    - A directory entry that associates a name with an existing file (inode).
    - Multiple hard links can point to the same inode. They share the same inode number and thus all metadata and data blocks.
    - The inode maintains a reference count (`ref` term or link count). Creating a hard link increments this count.
    - The file's data is only deleted when the reference count drops to zero (i.e., all hard links are removed).
    - Cannot span filesystems. Cannot link to directories (usually, to prevent loops).

- **Symbolic Link (Soft Link)**:
    - A special type of file that contains a path (text string) to another file or directory.
    - It has its own inode (different inode number from the target).
    - If the original file is deleted, the soft link becomes a "dangling" or "broken" link, as it points to a non-existent path.
    - Can span filesystems. Can link to directories.

Disk Partitioning & Filesystem Creation

1. List block devices: `lsblk`.
2. List current disk partitions: `fdisk -l {device_name}` or `parted -l`.
3. Create new partition: `fdisk {device_name}` (then `n` for new, configure sectors, `w` to write) or `parted`.

Sold to
cxl0603@gmail.com

4. Create filesystem on new partition: `mkfs.{fstype} {partition_name}` (e.g., `mkfs.ext4 /dev/sda1`).
5. Create a mount point directory: `mkdir /mnt/mydisk`.
6. Mount the partition: `mount {partition_name} {directory_name}` (e.g., `mount /dev/sda1 /mnt/mydisk`).
    - To make it permanent, add an entry to `/etc/fstab`.

RAID (Redundant Array of Independent Disks)

- A technology to combine multiple physical disk drives into one or more logical units for data redundancy, performance improvement, or both.
- **RAID 0 (Striping)**:
    - Data is split (striped) across multiple disks.
    - Purpose: Performance boost (multiple disks handle I/O).
    - Min disks: 2. Capacity: 100% of total.
    - Fault Tolerance: None. If one disk fails, all data in the array is lost.
- **RAID 1 (Mirroring)**:
    - Data is written identically to two (or more) disks, creating a mirror.
    - Purpose: Fault tolerance.
    - Min disks: 2. Capacity: 50% of total (e.g., two 1TB drives = 1TB usable).
    - Fault Tolerance: Can survive failure of 1 disk. Slight performance drag on writes.
- **RAID 5 (Distributed Parity)**:
    - Data and parity information are striped across three or more disks.
    - Purpose: Balance of performance and fault tolerance.
    - Min disks: 3. Capacity: (n-1) * disk_size.
    - Fault Tolerance: Can survive failure of 1 disk. Missing data can be reconstructed from remaining data and parity. Write performance can be slower due to parity calculation.
- **RAID 6 (Double Parity)**:
    - Similar to RAID 5 but uses two independent parity blocks.
    - Purpose: Higher fault tolerance.
    - Min disks: 4. Capacity: (n-2) * disk_size.
    - Fault Tolerance: Can survive failure of 2 disks.
- **RAID 10 (1+0) (Stripe of Mirrors)**:

Sold to
cxl0603@gmail.com

- Combines RAID 1 (mirroring) and RAID 0 (striping). Data is mirrored first, then the mirrors are striped.
- Purpose: High performance and good fault tolerance.
- Min disks: 4 (in pairs). Capacity: 50% of total.
- Fault Tolerance: Can survive multiple disk failures, as long as not both disks in the same mirror fail.

- **Software RAID (mdadm)**:
  - Managed by the OS.
  - Commands: `mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sda1 /dev/sdb1`, `cat /proc/mdstat` (check status), `mdadm --detail /dev/md0`, `mdadm --add /dev/md0 /dev/sdc1` (add spare), `mdadm --fail /dev/md0 /dev/sda1`, `mdadm --remove /dev/md0 /dev/sda1`.

- **Hardware RAID**: Managed by a dedicated RAID controller card. Often offers better performance and features like battery-backed write cache. Vendor-specific tools for management (e.g., MegaCLI, storcli).

Filesystem Performance Tuning

- **ext4**:
  - Mount options: `noatime` (disables access time updates), `data=writeback` (less safe but faster), `barrier=0` (disables write barriers, risky without battery-backed cache), `nobh` (no buffer head for data).
  - `tune2fs` options: `journal_data_writeback` (writeback journal data), `^has_journal` (remove journal - very risky, data loss on crash).

- **XFS**:
  - Mount options: `noatime`, `attr2` (more efficient extended attributes), `inode64` (allows inodes anywhere, good for large filesystems), `logbsize=256k` (larger log buffer), `largeio`.
  - Tools: `xfs_info` (display info), `xfs_growfs` (grow filesystem), `xfs_fsr` (defragment).

---

5. Kernel Concepts

Sold to
cxl0603@gmail.com

System Calls

- The primary interface through which user-space programs request services from the operating system kernel. They are the only legal entry points into the kernel apart from exceptions and traps.
- **Why needed**:

  1. Provide an abstract hardware interface for user space.
  2. Ensure system security and stability by having the kernel act as a middleman, judging access legality based on permissions, users, etc..

- **Return Value**: Typically `long`. Zero usually indicates success, a negative value indicates an error (errno). `perror()` can translate this to a human-readable error.
- **How they work (x86 example using `int $0x80` / `syscall`)**:

  1. **User space preparation**:

     - The system call number is placed in a specific register (e.g., `%eax` for `int $0x80`).
     - Arguments to the system call are placed in other predefined registers (e.g., `%ebx`, `%ecx`, `%edx`, etc., or `%rdi`, `%rsi`, `%rdx` for 64-bit `syscall`).

  2. **Trap instruction**: The program executes a special instruction (`int $0x80` or `syscall`). This incurs a software interrupt/trap, leading to an exception.
  3. **Mode Switch & Context Save**:

     - The CPU switches from user mode to kernel mode. The Current Privilege Level (CPL) in the Code Segment (CS) selector changes from 3 (user) to 0 (kernel).
     - The hardware (or kernel entry code) saves the user mode context: user stack pointer (`%esp` / `%rsp`), user instruction pointer (`%eip` / `%rip`), user code segment (`%cs`), user stack segment (`%ss`), and CPU flags (`eflags` / `rflags`) are typically pushed onto the kernel stack.
     - The kernel stack segment is retrieved from the Task State Segment (TSS). Linux typically has one TSS per CPU.

  4. **Locate Handler**:

     - The interrupt/trap number (e.g., `0x80`) is used as an index into the Interrupt Descriptor Table (IDT) to find the address of the kernel's system call handler routine. The IDT is pointed to by the `IDTR` register.

  5. **Execute Handler**:

Sold to
cxl0603@gmail.com

    - The kernel's system call dispatcher (`system_call` or `entry_SYSCALL_64`) uses the system call number (from `%eax` / `%rax`) to look up the specific kernel function in the `sys_call_table` .
    - The kernel verifies parameters for validity and legality.
    - The requested kernel function is executed.

6. **Return to User Space**:

    - The result of the system call is typically placed in `%eax` / `%rax` .
    - The CPL in CS is changed back from 0 to 3.
    - An instruction like `iret` (interrupt return) or `sysret` is executed. This restores the saved user registers (CS, EIP, SS, ESP, eflags) from the kernel stack.
    - Execution resumes in user mode at the instruction following the trap.

- **Why not call kernel functions directly?**

    - User mode and kernel mode operate on different stacks, so arguments can't be passed on the user stack for kernel functions; registers are used instead.
    - Kernel code resides in a different code segment, so a simple jump/call instruction from user mode won't work due to privilege levels and memory protection.

- **Registering a new System Call**:

    1. Define the system call function in the kernel source (e.g., under `kernel/`), include `linux/kernel.h` .
    2. Add an entry to the system call table (e.g., `/arch/{arch}/entry/syscalls/syscall_64.tbl`) with a unique number, ABI, and function name.
    3. Declare the system call function prototype in the system call header file (e.g., `include/linux/syscalls.h`).
    4. Add the new source file to the appropriate `Makefile` .
    5. Recompile the kernel.

Interrupts and Exceptions

- **Interrupts**: Asynchronous signals, usually from hardware, indicating an event that needs immediate attention (e.g., network card received a packet, disk I/O completed). They enable hardware to send signals to the processor. Distinguished by interrupt values (IRQ lines).
- **Exceptions**: Synchronous events generated by the processor during instruction execution due to an error or an abnormal condition that the kernel must handle.

    - **Faults**: Errors like page faults, division by zero. The instruction pointer generally points to

Sold to
cxl0603@gmail.com

the faulting instruction, allowing potential recovery/retry.
    - **Traps**: Intentional exceptions, like system calls (`int 0x80` , `syscall`) or breakpoint instructions (`int 3`). The instruction pointer points to the instruction after the trap.
    - **Aborts**: Severe, unrecoverable errors (e.g., hardware failure).

- **Difference**: Interrupts are asynchronous to the processor clock; exceptions are synchronous.
- **Interrupt Descriptor Table (IDT)**: A table used by the processor to determine the correct handler (Interrupt Service Routine - ISR) for each interrupt and exception. The base address and length of the IDT are stored in the `IDTR` register. Triggered by hardware interrupts, software interrupts (like `int n`), and processor exceptions.

Interrupt Handling

- **Interrupt Handler (ISR - Interrupt Service Routine)**: A kernel function executed in response to a specific interrupt. Each device generating interrupts has an associated handler, typically part of its device driver.
- **Interrupt Context**: ISRs run in a special, restricted context called interrupt context.
    - **Cannot Sleep**: Code in interrupt context cannot call functions that might sleep (e.g., request a semaphore, allocate memory that might block). This is because there's no backing process context to schedule when it wakes up, and sleeping would block further interrupt processing or the interrupted process.
    - Interrupt handlers often share the stack with the interrupted process (though dedicated interrupt stacks can be configured).

- **Interrupt Control**:
    - Disabling interrupts for the current processor or masking specific IRQ lines for the entire system.
    - Disabling interrupts also disables kernel preemption on that CPU.

- **Processing Split (Top Halves and Bottom Halves)**: To minimize time spent with interrupts disabled, ISR work is often split:
    - **Top Half (Interrupt Handler)**: Executes immediately upon interrupt. Does time-critical work: acknowledges the hardware, copies data to memory, schedules the bottom half, and exits quickly. Delays here can cause buffer overruns (e.g., network card dropping packets).
    - **Bottom Half**: Performs the bulk of non-time-sensitive processing deferred from the top half. Runs later with interrupts enabled. Implementations:
        1. **Softirqs (Software Interrupts)**: Static, compile-time defined. Used for the most

Sold to
cxl0603@gmail.com

timing-critical bottom-half processing (e.g., networking, block devices). Can run on return from hardware interrupt, in `ksoftirqd` threads, or other code checking for pending softirqs. They have priorities.
    2. **Tasklets**: Dynamically created, built on top of two softirqs (`HI_SOFTIRQ` , `TASKLET_SOFTIRQ`). Simpler to use. Only one tasklet of a given type runs at a time (serialized per type).
    3. **Work Queues**: Work is queued to kernel threads (worker threads) that run in process context. Therefore, work queue functions *can* sleep if necessary. Implemented based on kernel threads.

- **Choosing Implementation**:
    - Interrupt Handler (Top Half): Quick, time-sensitive work.
    - Softirq: Highly threaded, high-frequency, timing-critical bottom-half work.
    - Tasklet: Simpler, serialized bottom-half work.
    - Work Queue: If the deferred work needs to sleep or run in process context.
- **`ksoftirqd`** : A per-CPU kernel thread that runs at low priority (nice value 19) to handle softirq processing if there's a high load of softirqs, preventing them from starving user processes or hogging interrupt return paths.

Kernel Synchronization Methods

Essential for protecting shared data in a multi-processor environment or between interrupt handlers and process context. 1. **Atomic Operations**: Operations guaranteed to execute indivisibly without interruption. * Kernel provides interfaces for atomic operations on integers (using special type `atomic_t` to prevent compiler optimizations) and individual bits. * Examples: `atomic_inc()` , `atomic_dec_and_test()` , `test_and_set_bit()` . 2. **Spin Locks**: * A lock that causes a thread trying to acquire it to "spin" in a tight loop (busy-wait) until the lock becomes available. * Suitable for very short critical sections where the overhead of sleeping and waking a thread would be greater than the time spent spinning. * In Linux, spin locks are not recursive; a thread holding a spin lock attempting to acquire it again will deadlock. * Local interrupts (on the same CPU) must be disabled *before* acquiring a spin lock to prevent deadlock if an interrupt handler on the same CPU tries to acquire the same lock. * Cannot be held by code that might sleep. Use in interrupt context is common. 3. **Semaphores**: * Sleeping locks. If a thread tries to acquire an unavailable semaphore, it's placed on a wait queue and put to sleep (yielding the CPU). * Suitable for longer critical sections where busy-waiting would waste CPU. * Cannot be used in interrupt context because ISRs cannot sleep. * Can allow an arbitrary number of simultaneous lock holders (counting semaphores) or just one (binary semaphores). 4. **Mutexes (Mutual Exclusion)**: * Similar to a binary semaphore (allows only one lock holder at a time). * Also sleeping locks; threads attempting to acquire a locked mutex will sleep. * Cannot be used in

Sold to
cxl0603@gmail.com

interrupt context. * The PID of a thread in the waiting queue is stored in the mutex data structure. When freed, a waiting thread is woken up. * **Choosing a Method**: * Low overhead, short lock hold time: Spin lock. * Long lock hold time, need to sleep while holding lock: Semaphore/Mutex. * Locking in interrupt context: Spin lock.

`/proc` Filesystem

- A virtual filesystem that provides an interface to kernel data structures and runtime information.
- Files in `/proc` are not real files on disk but are generated on-the-fly when accessed.
- Examples: `/proc/cpuinfo`, `/proc/meminfo`, `/proc/interrupts`, `/proc/<PID>/` (for process-specific info).

Kernel Panic vs. Kernel Oops

- **Kernel Oops**:
  - A deviation from correct kernel behavior. When the kernel detects an invalid operation or condition, it prints an "oops message" (call trace, register dump) to the console/logs and typically kills the offending process.
  - A kernel process killed due to an oops might not release resources properly, potentially affecting other parts of the kernel since they share the same address space.
  - After an oops, the kernel's stability is compromised; it may be unreliable.
  - Multiple oops or an oops in critical code can lead to a kernel panic.
- **Kernel Panic**:
  - A fatal, unrecoverable error from which the kernel cannot safely continue running the OS.
  - The system halts to prevent further damage or data corruption.
  - It may dump a copy of kernel memory (a "crash dump" or "vmcore") to disk for later debugging.
  - The system then typically waits for a manual reboot or initiates an automatic reboot if configured.

---

6. Network Concepts

OSI Model (7 Layers)

A conceptual framework for network communication: 1. **Application Layer**: High-level APIs, user-

Sold to
cxl0603@gmail.com

facing protocols (e.g., HTTP, FTP, SMTP, Telnet). 2. **Presentation Layer**: Data translation, encryption, compression (e.g., SSL/TLS, ASCII, JPEG). 3. **Session Layer**: Manages connections (sessions) between applications (e.g., NetBIOS, RPC). 4. **Transport Layer**: End-to-end data transfer, reliability, flow control (e.g., TCP, UDP). 5. **Network Layer**: Logical addressing (IP addresses), routing, path determination (e.g., IP, ICMP, OSPF). 6. **Data Link Layer**: Physical addressing (MAC addresses), framing, error detection for a single link (e.g., Ethernet, PPP, Wi-Fi). 7. **Physical Layer**: Transmission of raw bit stream over physical medium (e.g., cables, radio waves, hubs).

TCP/IP Model (4 Layers - Simplified)

1. **Application Layer**: Combines OSI Application, Presentation, Session (e.g., HTTP, FTP, SMTP, DNS, Telnet).
2. **Transport Layer**: Same as OSI Transport Layer (e.g., TCP, UDP).
3. **Network/Internet Layer**: Same as OSI Network Layer (e.g., IP, ICMP, ARP).
4. **Data Link/Network Access Layer**: Combines OSI Data Link and Physical Layers (e.g., Ethernet drivers, Wi-Fi).

Key Protocols

- **TCP (Transmission Control Protocol)**:
  - Connection-oriented, reliable, stream-based transport protocol.
  - **Connection Setup (Three-Way Handshake)**:
    1. Client sends `SYN` (synchronize sequence numbers) to server.
    2. Server receives `SYN`, replies with `SYN-ACK` (synchronize + acknowledge).
    3. Client receives `SYN-ACK`, sends `ACK` (acknowledge) to server. Connection established.
  - **Data Transfer**: Data is segmented, sequence numbers used for ordering, acknowledgments for reliability.
  - **Connection Termination (Four-Way Handshake)**:
    1. Side A sends `FIN` (finish) to Side B.
    2. Side B sends `ACK` for Side A's `FIN`.
    3. Side B sends its own `FIN` to Side A (after sending any pending data).
    4. Side A sends `ACK` for Side B's `FIN`. Connection closed.
- **UDP (User Datagram Protocol)**:
  - Connectionless, unreliable, datagram-based transport protocol.

Sold to
cxl0603@gmail.com

- No handshake, no acknowledgments, no retransmissions at the UDP layer. Faster, less overhead.
- Used for applications where speed is critical and some data loss is acceptable (e.g., DNS, VoIP, online gaming).

- **IP (Internet Protocol)**:
  - Network layer protocol responsible for logical addressing (IP addresses) and routing packets across networks. Connectionless and unreliable (best-effort delivery).

- **ICMP (Internet Control Message Protocol)**:
  - Network layer protocol used for diagnostic and control purposes, and error reporting.
  - Not used to transport application data.
  - Does not use TCP or UDP; no ports are involved, and no connection is formed.
  - Used by utilities like `ping` (Echo Request/Reply) and `traceroute`.

Network Card Packet Reception

1. Network card receives packets from the network medium.
2. Card issues a hardware interrupt to the CPU.
3. Kernel responds by executing the network card's registered interrupt handler (top half).
4. The interrupt handler acknowledges the hardware, copies network packets from the card's buffer into main memory (e.g., into `sk_buff` structures), and removes them from the card's buffer.
5. Delays in this copying can lead to buffer overruns on the card, resulting in dropped packets.
6. Further processing (e.g., IP layer, TCP/UDP layer) is typically deferred to a bottom half (softirq).

DNS (Domain Name System)

- A hierarchical distributed naming system that translates human-readable domain names (e.g., `www.google.com`) into numerical IP addresses (e.g., `172.217.16.36`).
- Uses UDP for queries (typically port 53), but can use TCP for large responses (e.g., zone transfers).
- **DNS Resolution Process**:
  1. **Local Cache**: Application/OS first checks local DNS cache (e.g., browser cache, OS resolver cache, `/etc/hosts` file).
  2. **Resolver Server**: If not in local cache, query is sent to a configured DNS resolver (usually provided by ISP or a public resolver like Google's 8.8.8.8 or Cloudflare's 1.1.1.1).

Sold to
cxl0603@gmail.com

3. **Resolver Cache**: Resolver checks its own cache.
4. **Recursive Query (if not cached by resolver)**:
    - Resolver queries a **Root DNS Server** (there are 13 logical root server clusters worldwide).
    - Root server responds with the address of the appropriate **Top-Level Domain (TLD) Server** (e.g., for `.com`, `.org`, `.net`).
    - Resolver queries the TLD server.
    - TLD server responds with the address of the **Authoritative Name Server(s)** for the specific domain. There might be several layers of authoritative servers.
    - Resolver queries the authoritative name server.
    - Authoritative name server returns the IP address (A record for IPv4, AAAA for IPv6) or other requested record type.
5. **Response to Client**: Resolver caches the result (respecting TTL) and returns the IP address to the originating client.
6. Client uses the IP address to connect to the target server.

Routing Table

- A table stored in a router or networked computer that lists the routes to particular network destinations.
- `route -n` or `ip route show` displays the kernel routing table.
- **Fields**:
    - **Destination**: Target network or host IP address. `0.0.0.0` (or `default`) is the default gateway entry.
    - **Gateway (Router/Next Hop)**: IP address of the next-hop router to reach the destination. `0.0.0.0` means the destination is directly connected (on the same network segment).
    - **Genmask (Netmask)**: Subnet mask for the destination network. `255.255.255.255` for a host route, `0.0.0.0` for the default gateway entry.
    - **Flags**: Indicate route properties (e.g., `U` =Up, `G` =Gateway, `H` =Target is a host).
    - **Metric**: Cost or distance to the destination. Lower is better. Number of hops for some protocols.
    - **Iface**: Network interface through which packets for this route will be sent (e.g., `eth0`, `wlan0`).

Network Troubleshooting Commands

- **`ping`**:

Sold to
cxl0603@gmail.com

- Tests network connectivity to a host using ICMP Echo Request/Reply messages.
- Usage: `ping {ipaddr_or_hostname}`.
- Options: `-c <count>` (number of pings), `-s <size>` (packet size).
- Reports packet loss and round-trip times.

- **`telnet`**:
  - User interface to the TELNET protocol, but often used to check TCP port accessibility on a remote server.
  - Usage: `telnet {ipaddr_or_hostname} {port}`.
  - Mechanism: Based on TCP. User data and Telnet control info are sent over an 8-bit byte-oriented data connection.
  - If connection fails, the remote port might not be listening, or a firewall might be blocking it.

- **`traceroute` (Linux) / `tracert` (Windows)**:
  - Shows the path (sequence of routers) packets take to a network host and measures transit delays.
  - Usage: `traceroute {ipaddr_or_hostname}`.
  - Works by sending packets with increasing TTL (Time-To-Live) values.

- **`netstat`**:
  - Displays network connections (TCP, UDP, raw), routing tables, interface statistics, masquerade connections, multicast memberships, etc..
  - Common options:
    - `netstat -tulnp`: Show TCP/UDP listening ports and associated programs.
    - `netstat -rn`: Show routing table (like `route -n`).
    - `netstat -i`: Show interface statistics.

- **`ss`**: Utility to investigate sockets. Can display more information and is generally faster than `netstat`.
  - `ss -tulnp`: Similar to `netstat -tulnp`.
  - `ss -s`: Summary statistics.

- **`ip` (from iproute2 package)**: Modern replacement for `ifconfig`, `route`, `arp`.
  - `ip addr show`: Display IP addresses and network interfaces.
  - `ip link show`: Display interface information.
  - `ip route show`: Display routing table.

- **`nmap`**: Powerful network scanning tool. Can discover hosts, services, OS versions, etc.

Sold to
cxl0603@gmail.com

- `tcpdump` / `wireshark` : Packet sniffing tools for detailed network traffic analysis.

Workflow: Typing a URL in Browser

1. **URL Parsing**: Browser parses the URL (protocol, domain name, path).
2. **DNS Lookup**:
    - Browser/OS checks local DNS cache.
    - If not found, queries DNS resolver, which may recursively query root, TLD, and authoritative servers to get the IP address for the domain name.
3. **TCP Connection (for HTTP/HTTPS)**:
    - Browser initiates a TCP connection to the server's IP address on the appropriate port (80 for HTTP, 443 for HTTPS). This involves the three-way handshake (SYN, SYN-ACK, ACK).
4. **HTTP/HTTPS Request**:
    - Browser sends an HTTP(S) request (e.g., `GET /path HTTP/1.1`) to the server. For HTTPS, a TLS handshake occurs first to establish a secure channel.
5. **Server Processing**: Web server processes the request (e.g., fetches a file, runs a script).
6. **HTTP/HTTPS Response**: Server sends an HTTP(S) response back to the browser (status code, headers, content).
7. **Rendering**: Browser renders the received content (HTML, CSS, JavaScript). May make further requests for embedded resources (images, scripts).
8. **TCP Connection Teardown**: Connection is closed (four-way handshake).

Packet Journey (Simplified): HTTP GET request packet → TCP layer adds TCP header → IP layer adds IP header (using routing table for destination) → Data Link layer adds Ethernet header (MAC address) → Physical layer transmits frames.

---

7. Containers & Virtualization

Linux Containers

Lightweight OS-level virtualization. Applications run in isolated environments on a shared kernel.

Container Technology Stack Container Applications → Container Runtime (Docker, Podman) → Container Engine (containerd, CRI-O) → Linux Kernel Features.

Sold to
cxl0603@gmail.com

Core Kernel Features for Containers: 1. **Namespaces**: Provide isolation of system resources for processes. * Key feature: Isolate processes from each other. One of the foundational technologies for containers, used for resource segregation. * **Types**: * **PID Namespace**: Isolates process IDs. The first process in a new PID namespace gets PID 1. A child process can have its own PID namespace, being PID 1 within it, while also having a different PID in its parent's namespace. * **Mount Namespace**: Isolates filesystem mount points. Processes in different mount namespaces can have different views of the filesystem hierarchy. * **UTS Namespace**: Isolates hostname and domain name. * **IPC Namespace**: Isolates inter-process communication resources (System V IPC, POSIX message queues). * **User Namespace**: Isolates user and group IDs. Allows a process to have root privileges within its namespace without having them on the host system. * **Network Namespace**: Isolates network devices, IP addresses, routing tables, port numbers. Each network namespace has its own network stack. * **Time Namespace**: Isolates system time. * **Investigation**: `ls -la /proc/<PID>/ns/` lists namespaces for a process. `nsenter -t <PID> -n -p` to enter network and PID namespace of another process. 2. **Control Groups (Cgroups)**: * Linux kernel feature to limit, account for, and isolate resource usage (CPU, memory, disk I/O, network) of a collection of processes. * Key for containers, as multiple processes often run within a container and need to be controlled together. Used in Kubernetes for pod resource requests/limits. * **Features**: * **Resource Limiting**: Configure a cgroup to limit resource usage (e.g., memory, CPU). * **Prioritization**: Control relative access to resources (CPU, disk, network) compared to processes in other cgroups. * **Accounting**: Monitor and report resource usage at the cgroup level. * **Control**: Change status (frozen, stopped, restarted) of all processes in a cgroup with a single command. * **Hierarchy**: Cgroups are organized hierarchically in `/sys/fs/cgroup/` (e.g., `/sys/fs/cgroup/memory/`, `/sys/fs/cgroup/cpu/`). * **Management**: Check process cgroup: `cat /proc/<PID>/cgroup`. View/set limits: e.g., `cat /sys/fs/cgroup/memory/docker/<CONTAINER_ID>/memory.limit_in_bytes`. 3. **Capabilities**: Divide root privileges into distinct units. Containers can be granted only necessary capabilities, reducing risk if compromised. 4. **Seccomp (Secure Computing Mode)**: Filters system calls a process can make, restricting kernel attack surface. 5. **AppArmor / SELinux**: Mandatory Access Control systems that can further confine containerized applications.

Container Network Interface (CNI) * A standard for connecting containers to networks. Defines how network interfaces are created for containers and how they are attached to networks. * Plugins (e.g., Calico, Flannel, Cilium) implement the CNI specification.

Virtualization (Hardware Virtualization)

- Creates Virtual Machines (VMs), each with its own full OS, kernel, and virtualized hardware.
- **Structure**: Guest Application → Guest OS → Hypervisor/VMM (Virtual Machine Monitor) → Host OS (Type 2) or Hardware (Type 1) → Hardware Resources.
- **Hypervisor (VMM)**: Software that creates and runs VMs.

Sold to
cxl0603@gmail.com

- **Type 1 (Bare-metal)**: Runs directly on host hardware (e.g., VMware ESXi, Xen, Hyper-V).
- **Type 2 (Hosted)**: Runs as an application on top of a host OS (e.g., VirtualBox, VMware Workstation/Fusion).

- **CPU Virtualization**:
  - **Trap-and-Emulate**: Sensitive (privileged) instructions executed by the Guest OS are trapped by the VMM. The VMM then emulates the behavior of the instruction on the virtual hardware.
  - Hardware-assisted virtualization (Intel VT-x, AMD-V) provides extensions to make this more efficient.

- **Memory Virtualization**:
  - VMM maintains mappings from Guest Virtual Addresses (GVA) to Guest Physical Addresses (GPA), and then from GPA to Host Physical Addresses (HPA). GVA → GPA → HPA.
  - **Shadow Page Tables**: The VMM can create shadow page tables that directly map GVA to HPA to avoid two levels of translation for each memory access.
    - When Guest OS tries to modify its page tables (e.g., change CR3), VMM intercepts this, updates the shadow page tables, and sets a virtual CR3 pointing to the shadow tables.
    - Separate shadow page tables can be used for guest user space and guest kernel space.
  - Hardware assistance like Intel EPT (Extended Page Tables) and AMD RVI (Rapid Virtualization Indexing) / NPT (Nested Page Tables) allow hardware to handle the GVA → HPA translation directly, improving performance.

Containers vs. VMs

- **Virtualization Level**:
  - Containers: Virtualize the OS (share host kernel).
  - VMs: Virtualize the hardware (each VM has its own kernel).

- **Hypervisor**:
  - Containers: Do not use a hypervisor (rely on kernel features).
  - VMs: Require a hypervisor.

Sold to
cxl0603@gmail.com

- **Kernel**:
  - Containers: Share the host OS kernel.
  - VMs: Each VM has its own independent kernel.
- **Isolation**: VMs generally offer stronger isolation than containers because of separate kernels.
- **Overhead**: Containers are much more lightweight (less resource consumption, faster startup) than VMs.
- **Density**: More containers can run on a host than VMs.

---

8. System Performance & Monitoring

Performance Analysis Methodology

- **USE Method (Brendan Gregg)**: For every system resource (CPU, memory, disk, network, etc.):
  1. **U**tilization: How busy is the resource? (Percentage of time it's working).
  2. **S**aturation: Is there a queue of work waiting for the resource? (e.g., run queue length for CPU, I/O queue for disk).
  3. **E**rrors: Are there any errors occurring? (e.g., disk I/O errors, network packet drops).
- **RED Method (Tom Wilkie)**: For every service/application:
  1. **R**ate: Number of requests per second.
  2. **E**rrors: Number of failed requests per second.
  3. **D**uration: Response time distribution (latency) for requests.

System Monitoring Tools

- `vmstat` **(Virtual Memory Statistics)**: Reports virtual memory, processes, I/O, CPU activity.
  - `vmstat <interval> <count>` (e.g., `vmstat 1 5`).
  - **Columns**:
    - `procs`:
      - `r`: Runnable processes (waiting for CPU or running). If `r` > #CPU cores, indicates CPU saturation.
      - `b`: Processes in uninterruptible sleep (blocked, usually for I/O). High `b` can indicate I/O bottleneck or slow disks.

Sold to
cxl0603@gmail.com

- `memory`:
  - `swpd`: Amount of virtual memory used (swap space used, in KB).
  - `free`: Amount of idle/free physical memory (KB).
  - `buff`: Memory used as buffer cache (for block device I/O metadata, KB).
  - `cache`: Memory used as page cache (for file contents, KB).
  - `inact`: Inactive memory (KB). High inactive memory might signal a leak.
  - `active`: Active memory (KB), constantly accessed.
- `swap`:
  - `si`: Kilobytes swapped IN from disk per second (page-ins). Should ideally be 0.
  - `so`: Kilobytes swapped OUT to disk per second (page-outs). Should ideally be 0. >10 blocks/sec is problematic.
- `io`: (Blocks per second, typically 1KB blocks)
  - `bi`: Blocks received from block devices (read).
  - `bo`: Blocks sent to block devices (write).
- `system`:
  - `in`: Number of interrupts per second (including clock). High `in` means CPU is busy with interrupts.
  - `cs`: Number of context switches per second. High `cs` can indicate inefficient memory use or lock contention.
- `cpu`: Percentage of total CPU time.
  - `us`: User CPU time (running non-kernel code).
  - `sy`: System (kernel) CPU time. If `sy` > `us`, system spends more time in kernel than on real work.
  - `id`: Idle CPU time.
  - `wa`: CPU time spent waiting for I/O. >20% can indicate I/O wait issues. If `wa` is high, it doesn't necessarily mean I/O performance problem, just that CPU is idle waiting for I/O.
  - `st`: Stolen time (by hypervisor for other VMs).

- **Interpretation**: `r` > CPU cores implies CPU bottleneck. Consistent `b` > 0 implies I/O bottleneck. `si` / `so` > 0 implies memory pressure.

- **`iostat` (I/O Statistics)**: Reports CPU and I/O statistics for devices and partitions.

Sold to
cxl0603@gmail.com

- `iostat -x <interval> <count>` (extended stats).
- **Key Metrics (per device)**:
  - `r/s`, `w/s`: Read/write operations (IOPS) per second.
  - `rkB/s`, `wkB/s`: Kilobytes read/written per second (throughput).
  - `rrqm/s`, `wrqm/s`: Read/write requests merged per second.
  - `r_await`, `w_await`: Average time (ms) for read/write requests to be served (includes queue time + service time).
  - `await`: Average time (ms) for any I/O request to be served.
  - `aqu-sz` (avgqu-sz): Average queue length (number of requests waiting).
  - `rareq-sz`, `wareq-sz`: Average request size (KB) for reads/writes.
  - `svctm`: Average service time (ms) for I/O requests (deprecated, can be misleading).
  - `%util`: Percentage of CPU time during which I/O requests were issued to the device (device utilization). >80% often indicates saturation.
- **Performance Thresholds**: `await` < 1ms (good SSD), < 10ms (acceptable SSD); < 20ms (good HDD), < 50ms (acceptable HDD). `%util` > 80% indicates saturation.

- **`top` / `htop` (Process Monitoring)**: Display dynamic real-time view of running system processes.
  - **Header Info**:
    - Uptime, number of users, load average (1, 5, 15 min).
    - Tasks: Total, running, sleeping, stopped, zombie.
    - CPU states (%): `us` (user), `sy` (system), `ni` (nice), `id` (idle), `wa` (IO-wait), `hi` (hardware irq), `si` (software irq), `st` (stolen).
    - Memory: Physical (total, used, free, buffers), Swap (total, used, free, cached).
  - **Process List Columns (common)**: PID, USER, PR (priority), NI (nice value), VIRT (virtual memory), RES (resident memory), SHR (shared memory), S (status), %CPU, %MEM, TIME+ (CPU time), COMMAND.
  - **Interactive Commands (top)**: `P` (sort by CPU), `M` (sort by MEM), `T` (sort by TIME), `k` (kill), `r` (renice), `f` (fields), `1` (toggle individual CPU core stats).
  - **Load Average**: Average number of processes in the run queue (R) or waiting for disk I/O (D) over 1, 5, and 15 minutes.
    - < 1.0 per core: Underutilized.
    - = 1.0 per core: Fully utilized.
    - > 1.0 per core: Overloaded, tasks are queuing.

Sold to
cxl0603@gmail.com

        - High load with low CPU utilization can indicate I/O bottleneck (many D state processes).
    - **Memory Usage in top**: VIRT, RES, SHR, %MEM.

- **`sar` (System Activity Reporter)**: Collects, reports, and saves system activity information. Part of `sysstat` package.
    - `sar -u 1 10` : CPU utilization every second, 10 times.
    - `sar -r 1 10` : Memory and swap usage.
    - `sar -b 1 10` : Block device I/O statistics (tps, rtps, wtps, bread/s, bwrtn/s).
    - `sar -q 1 10` : Queue length and load averages.
    - `sar -n DEV 1 10` : Network interface statistics.
    - `sar -P ALL 1 10` : Per-CPU statistics.
    - Reads historical data from `/var/log/sa/saDD` files (e.g., `sar -u -f /var/log/sa/sa01`).

- **`free`** : Displays amount of free and used physical and swap memory, and buffers/cache used by kernel.
    - `free -h` (human-readable).
    - Low `buffers/cache` free memory or low `swap free` may indicate need for memory upgrade.
    - "available" column in newer `free` versions is a better estimate of memory available for new applications.

- **`pidstat` (Process Statistics)**: Reports statistics for Linux tasks.
    - `pidstat -u 1 <PID>` : CPU usage for a specific PID.
    - `pidstat -r 1 <PID>` : Memory usage for a PID.
    - `pidstat -d 1 <PID>` or `pidstat -d -p <PID>` : I/O statistics for a PID (kB*rd/s, kB*wr/s, IODelay).
    - `pidstat -w 1` : Context switch statistics per process.

- **`mpstat -P ALL 1`** : Per-CPU utilization statistics.

- **`lscpu`** : Displays CPU architecture information (cores, sockets, NUMA).

- **`numactl --hardware`** : Shows NUMA topology.

- **`cpupower frequency-info`** : Current CPU frequency settings.

CPU Performance Tuning

Sold to
cxl0603@gmail.com

- **CPU Scaling Governors**: Control how CPU frequency is managed.
  - `cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor` to view current governor.
  - `echo performance > .../scaling_governor` to set to performance mode (runs at max frequency).
  - Governors: `performance`, `powersave`, `ondemand` (dynamic, scales with load), `conservative` (slower scaling than ondemand), `userspace`.
- **CPU Affinity**: Bind processes to specific CPU cores using `taskset`.
  - `taskset -c 0,1 ./myprocess` (bind to CPUs 0 and 1).
  - `taskset -p <affinity_mask> <PID>` (set affinity for running process).
- **Process Priorities**: Use `nice` / `renice` for normal processes, `chrt` for real-time scheduling policies (FIFO, RR).

---

9. Advanced Troubleshooting Scenarios

Scenario 1: High System Load with Low CPU Usage

- **Symptoms**: High load average (e.g., >10-15), high CPU idle time (>70%), system sluggish.
- **Cause**: Often due to processes in uninterruptible sleep (D state), typically waiting for slow I/O (disk, NFS).
- **Investigation Steps**:
  1. `vmstat 1`: Check `r` (runnable) and `b` (blocked) columns.
  2. `ps aux | awk '$8~/^D/'` or `ps -eo pid,stat,comm | grep "D"`: Find processes in D state.
  3. `iostat -x 1`: Analyze disk I/O (look for high `%util`, high `await`, low throughput).
  4. For D-state PIDs:
     - `cat /proc/<PID>/wchan`: Show what kernel function the process is sleeping in.
     - `lsof -p <PID>`: List open files (to see what it might be accessing).
  5. `strace -p <PID>` (if possible, D state often non-responsive to strace): Try to see syscalls.
  6. `iotop -o`: Identify I/O-heavy processes.

Sold to
cxl0603@gmail.com

- **Common Root Causes**:
  - **Storage Hardware Problems**: Failing disks. Check `dmesg | grep -i "I/O error"`, `smartctl -a /dev/sdX`.
  - **Memory Pressure / Swap Thrashing**: `vmstat` (si/so columns), `free -h`. Identify memory hogs with `ps aux --sort=-%mem`.
  - **NFS Issues**: Slow NFS server or network problems.
  - **Too many interrupts**: `vmstat` (system.in column), `top` (hi/si).

Scenario 2: Database Performance Degradation

- **Symptoms**: Queries significantly slower than normal.
- **Troubleshooting Approach**:
  - **Phase 1: System Resource Analysis (on DB server)**
    - CPU: `top -p $(pgrep mysqld)`, `mpstat -P ALL 1`. Check for saturation or high `wa` time.
    - Memory: `ps -p $(pgrep mysqld) -o ...`, `cat /proc/$(pgrep mysqld)/status | grep Vm`. Check for swapping or low available memory.
    - I/O: `pidstat -d 1 -p $(pgrep mysqld)`, `lsof -p $(pgrep mysqld) | grep -E "\.ibd|\.MYD"`. Check `await` and `%util` for DB storage devices.
  - **Phase 2: Database-Specific Investigation**
    - Connections: `ss -tuln | grep <DB_PORT>` (e.g., 3306 for MySQL).
    - Lock Contention: `SHOW PROCESSLIST;` (MySQL), `SHOW ENGINE INNODB STATUS\G` (MySQL, check for deadlocks).
    - Slow Query Analysis: Enable slow query log. `SELECT * FROM information_schema.PROCESSLIST WHERE TIME > 10;` (MySQL).
  - **Phase 3: Storage Subsystem Analysis**
    - Identify DB data directories: `lsof -p $(pgrep mysqld) | grep data`.
    - Monitor I/O on DB files/devices: `iotop -p $(pgrep mysqld)`.
    - Test filesystem performance on DB mount points: `time dd if=/dev/zero of=<mount>/test_file bs=1M count=100 oflag=direct`.
  - **Other causes**: DB internal operations (migration, sync in distributed DBs). Check DB

Sold to
cxl0603@gmail.com

logs.

Scenario 3: Memory Exhaustion and OOM Killer

- **Symptoms**: Applications crashing, system unresponsive, "Out of memory" errors in logs.
- **Investigation**:

  1. **Immediate Assessment**: `free -h`, `cat /proc/meminfo`. Check `dmesg | grep -i "killed process"` or `journalctl | grep -i "oom"` for OOM killer activity. Check swap: `swapon -s`.
  2. **Process Analysis**: `ps aux --sort=-%mem | head -20` to find current memory hogs. Monitor memory usage over time for suspect processes.
  3. **System Memory Configuration**: Check `ulimit -a | grep memory`, `/proc/sys/vm/overcommit_memory`, `/proc/sys/vm/overcommit_ratio`. Check cgroup memory limits if used (`/sys/fs/cgroup/.../memory.limit_in_bytes`).
  4. **Analyze Fragmentation**: `cat /proc/buddyinfo`, `cat /proc/pagetypeinfo`.

- **Preventive Measures**:

  - Adjust `oom_score_adj` for critical processes: `echo -1000 > /proc/<PID>/oom_score_adj`.
  - Set up memory monitoring scripts to alert or log top memory consumers when usage exceeds a threshold.
  - Tune `vm.overcommit_memory` and `vm.overcommit_ratio`.
  - Increase RAM or swap, or optimize applications.

Scenario 4: Disk I/O Performance Problems

- **Symptoms**: Slow application response, high `wa` CPU time.
- **Investigation**:

  1. **System-Wide I/O**: `iostat -x 1 10` (look for high `%util`, `await`). `iotop -o` or `pidstat -d 1` for process-level I/O.
  2. **Device Analysis**: `iostat -x 1 3 -p <device>` for specific devices. Check queue depth (`/sys/block/sdX/queue/nr_requests`) and scheduler (`.../scheduler`).
  3. **Filesystem Analysis**: `lsof | awk '{print $9}' | ... | uniq -c | sort -nr | head -20` to find hot files. `df -h` (usage), `df -i` (inode usage). `find / -type f -size +<size>M` for large files. Check fragmentation (e.g.,

Sold to
cxl0603@gmail.com

`e4defrag -c /` for ext4).

4. **Application I/O Patterns**: `strace -p <PID> -e trace=read,write,open,close`. `ls -la /proc/<PID>/fd/` and `lsof -p <PID> | grep REG` for open files.

Troubleshooting General Failures (e.g., batched task failed on some machines)

- **Always check logs**: On executor and remote machines.
- **Probable causes**:
  - **Disk full on remote**: `df -lh`. Use `du` to find large directories/files.
  - **Disk failed on remote**: `fsck` to check/repair. `dmesg` for I/O errors.
  - **Insufficient memory on remote**: Check OOM killer logs (`dmesg | egrep -i 'killed process'`). Check current memory (`free`, `vmstat`, `top`).
  - **Missing dependency/file on remote**: Run script manually on that machine to see output.
  - **Network resource inaccessible from remote**: `ping` or `telnet` to test accessibility of required resources.
- **If `ls` is stuck**:
  - `strace ls` to see where it hangs.
  - Could be a directory mounted to an unavailable network address (e.g., NFS).
  - `mount -l` (or `cat /proc/mounts`) to see mount information.
  - `umount <directory>` to unmount if necessary.

NFS Troubleshooting

1. Login to NFS server and client.
2. **Client side**:
   - Can client reach server? `ping <server_ip_or_name>`.
   - If not, check local name service (`nisping -u` if using NIS), host info, or try from another client.
   - Can client contact NFS services on server? (RPC check) `rpcinfo -p <server_name>`.
   - Check if NFS shares are mounted: `df -hT`, `mount | grep nfs`.
3. **Server side**:

Sold to
cxl0603@gmail.com

- Are NFS services running? (`rpcinfo -p localhost` , `systemctl status nfs-server rpcbind`).
- Check `nfsd` and `mountd` daemons are running and responsive.
- Can server reach client? (less common issue for basic mount).
- Check `/etc/exports` for correct export options and client permissions.
- Check server logs for errors.

4. **Both sides**: Check firewalls. Check network configuration (`/etc/netmasks` , `/etc/nsswitch.conf`).
5. **Performance Issues**:
   - `iostat -x` on server to check for long `await` on disk serving NFS.
   - `lsof +D <NFS_mount_dir_on_server>` to see processes using the exported directory on server.
   - `pidstat -d -p <PID>` on server for I/O delay of relevant server processes.
   - `iotop` on server to see which process is saturating I/O.

10. Essential Linux Commands & Concepts

File System Navigation & Manipulation

- `ls` : List directory contents.
  - `ls -l` : Long format.
  - `ls -a` : Show hidden files.
  - `ls -R` : Recursive.
  - `ls -lh` : Long format, human-readable sizes.
- `cd` : Change directory.
- `pwd` : Print working directory.
- `mkdir` : Create directory.
- `rmdir` : Remove empty directory.
- `rm` : Remove files or directories (`rm -r` for recursive).
- `cp` : Copy files or directories (`cp -r` for recursive).
- `mv` : Move or rename files/directories.
- `find` : Search for files. E.g., `find /path -name "*.txt" -type f` .
- `grep` : Search for patterns in files. E.g., `grep "error" logfile.txt` .
- `cat` : Concatenate and display file content.
- `less` / `more` : Paginate through file content.

Sold to
cxl0603@gmail.com

- `head` / `tail` : Display beginning/end of files. `tail -f` to follow.
- `touch` : Create an empty file or update timestamps.
- `chmod` : Change file permissions.
- `chown` : Change file owner/group.
- `df` : Display disk space usage. `df -h` , `df -i` (inodes).
- `du` : Estimate file space usage. `du -sh /path` .

Process Management

- `ps` : Report a snapshot of current processes.
  - `ps aux` : Show all processes in BSD format.
  - `ps -ef` : Show all processes in System V format.
  - `ps -eo pid,ppid,stat,comm` : Custom format.
- `kill` : Send a signal to a process.
  - `kill <PID>` : Send `SIGTERM` (default).
  - `kill -9 <PID>` or `kill -SIGKILL <PID>` : Force kill.
  - `kill -HUP <PID>` or `kill -1 <PID>` : Send hangup signal (often to restart/reload config).
  - `kill -l` : List signal names.
- `pstree` : Display processes as a tree. `pstree -p -a` (show PIDs and command lines).
- `bg` / `fg` / `jobs` : Job control for shell.
- `nohup ./command &` : Run command immune to hangups, in background.
- `ulimit` : View or set resource limits for processes. `ulimit -a` (show all), `ulimit -u <num>` (limit max user processes). `/etc/security/limits.conf` for persistent limits.

System Information & Monitoring (Covered previously, quick list)

- `uname -a` : System information.
- `uptime` : System uptime and load averages.
- `dmesg` : Kernel ring buffer messages.
- `vmstat` , `iostat` , `top` / `htop` , `sar` , `free` , `pidstat` , `mpstat` .

Network

- `ifconfig` (legacy) / `ip addr show` : Network interface configuration.
- `route -n` (legacy) / `ip route show` : Routing table.

Sold to
cxl0603@gmail.com

- `ping`, `telnet`, `traceroute`, `netstat`, `ss`, `dig`, `nslookup`.
- `lsof -i :<port>`: List processes using a specific port.
- `tcpdump`: Capture network packets.

User Management

- `who`: Show who is logged on.
- `w`: Show who is logged on and what they are doing.
- `last`: Show last logged in users.
- `id`: Display user and group information.
- `su` / `sudo`: Switch user / execute command as superuser.
- `useradd`, `usermod`, `userdel`, `groupadd`, etc.

Shell Operations

- **Pipes (`|`)**: Redirects the standard output of one command to the standard input of another. E.g., `ls -l | grep ".txt"`.
- **Redirection**:
  - `>`: Redirect standard output to a file (overwrite).
  - `>>`: Redirect standard output to a file (append).
  - `<`: Redirect standard input from a file.
  - `2>`: Redirect standard error.
  - `&>`: Redirect both stdout and stderr.
- **Command Substitution**: `$(command)` or `` `command` ``. Output of `command` replaces the construct.
- **Variables**: `VARIABLE=value`, `echo $VARIABLE`. `$` references a variable. `$$` is the PID of the current shell.
- **Wildcards/Globbing**: `*` (any characters), `?` (any single character), `[]` (character class). Expansion is done by the shell *before* the command is executed.
- **What happens when you type `ls -l *.c` and hit Enter**:
  1. **Input**: Shell reads the command (e.g., using `getline()`).
  2. **Tokenization**: String is broken into tokens: `ls`, `-l`, `*.c`.
  3. **Expansion & Alias Check**: Shell checks for aliases and expands special characters/wildcards. `*.c` is replaced with all filenames ending in `.c` in the current directory. This involves reading directory contents (syscalls like `opendir`, `readdir`, `closedir`).
  4. **Built-in Check**: Shell checks if the first token (`ls`) is a built-in command. If so, it

Sold to
cxl0603@gmail.com

executes directly.

5. **PATH Lookup**: If not built-in, shell searches for an executable file named `ls` in directories listed in the `PATH` environment variable. `PATH` is tokenized (usually by `:`) and `/ls` is appended to each token to form full paths (e.g., `/bin/ls`, `/usr/bin/ls`).
6. **`fork()` and `execve()`**:
   - Shell calls `fork()` to create a new child process.
   - The child process calls `execve()` to execute `/path/to/ls` with the arguments (`-l` and expanded filenames). `execve()` replaces the child's memory image with that of `ls`.
7. **Parent Waits**: The parent shell process typically calls `wait()` or `waitpid()` to wait for the child process (`ls`) to complete.
8. **`ls` Execution**: `ls` reads file information from the filesystem (metadata from inodes, directory contents) and formats the output.
9. **Output**: `ls` writes its output to standard output (usually the terminal), via `write()` syscall.
10. **`exit()`**: `ls` (child process) calls `exit()` when done.
11. **Shell Resumes**: Parent shell resumes, prompt is displayed.

Debugging Tools

- **`strace`**: Traces system calls and signals.
  - `strace ./command`: Run command and trace its syscalls.
  - `strace -p <PID>`: Attach to a running process.
  - `strace -f`: Trace child processes forked by the traced process.
  - `strace -e trace=<syscall_set>`: Trace specific syscalls (e.g., `trace=file` for file ops, `trace=network` for network ops).
  - Output includes syscall name, arguments, and return value, printed to stderr or `-o <file>`.
- **`ltrace`**: Traces library calls made by a program. `ltrace -p <PID>`.
- **`gdb` (GNU Debugger)**: Powerful source-level debugger.
  - **How it works (using `ptrace`)**:
    - `ptrace()` is a system call that allows one process (tracer, e.g., GDB) to observe and control another process (tracee), examine/change its memory and registers.
    - `ptrace()` connects the two processes by setting fields in their `task_struct`.

Sold to
cxl0603@gmail.com

    - **For a new process**: GDB forks a child. The child calls `ptrace(PTRACE_TRACEME, ...)` to allow its parent (GDB) to trace it. Then the child calls `exec()` to load the program to be debugged.
    - **For an ongoing process (attach)**: GDB calls `ptrace(PTRACE_ATTACH, <PID>, ...)` to attach to the target process. The target becomes a tracee of GDB. GDB sends `SIGSTOP` to the target to halt it for inspection.
    - **Restrictions**: Cannot attach to self, cannot attach multiple times, cannot attach to `init`.

  - **Breakpoints**: GDB replaces the instruction at the breakpoint address with a software interrupt instruction (e.g., `INT 3` on x86, opcode `0xCC`). The original instruction is saved. When the tracee hits `INT 3`, it traps to the kernel, which notifies GDB (via `SIGTRAP` because of `ptrace`). GDB then replaces `INT 3` with the original instruction, can step the original instruction, and then re-inserts `INT 3` if needed. It may also set PC back one to re-execute the original instruction after user interaction.
  - **`next` / `step`**: GDB uses `ptrace` to single-step instructions or set temporary breakpoints to run until a certain line is reached.

File Descriptors (FD)

- A small non-negative integer that acts as an abstract handle (identifier) for an open file, socket, pipe, or other I/O resource within a process.
- Each process has its own file descriptor table.
- Standard FDs:
  - 0: Standard Input (stdin).
  - 1: Standard Output (stdout).
  - 2: Standard Error (stderr).

- `open()` returns a new FD. `read()`, `write()`, `close()` use FDs.
- View open FDs for a process: `ls -l /proc/<PID>/fd/` or `lsof -p <PID>`.
- **Kernel Representation of Open Files**:

  1. **Descriptor Table (per process)**: Indexed by FD, points to an entry in the File Table.
  2. **File Table (system-wide)**: Shared by all processes. Entries contain current file position/offset, status flags (e.g., append mode), reference count (how many FDs point to this entry), and a pointer to an entry in the v-node table.
  3. **V-node Table / Inode Table (system-wide)**: Shared by all processes. Each entry (inode) contains most of the file's metadata (from `stat` structure).

Sold to
cxl0603@gmail.com

ELF (Executable and Linkable Format)

- Common standard file format for executables, object code, shared libraries, and core dumps on Linux and other Unix-like systems.
- **Structure**:

  1. **ELF Header**: Contains metadata about the file: magic number, class (32/64-bit), type (executable, relocatable, shared), machine architecture (x86, ARM), entry point address (start address for execution).
  2. **Program Header Table (Segments)**: Describes zero or more segments. Relevant at runtime for loading the file into memory. Specifies how to create a process image.

     - Examples: Code segment (`.text`), Data segment (`.data` , `.bss`), dynamic linking information.
     - Segment attributes: Type (load, dynamic), offset in file, virtual address (Vaddr) to load to, physical address (Paddr, often ignored), file size, memory size (Memsz, can be > Filesz for BSS).

  3. **Section Header Table (Sections)**: Describes zero or more sections. Relevant at link time for linking object files. Contains information for linking and relocation.

     - Examples: `.text` (code), `.data` (initialized data), `.bss` (uninitialized data), `.rodata` (read-only data), symbol table, relocation information.

  4. **Data**: Actual data referred to by program headers or section headers.

- **Loading an ELF Executable (Simplified)**:

  1. Kernel reads ELF header and program header table.
  2. For each loadable segment (PT_LOAD), kernel maps the segment from the file into the process's virtual address space at the specified Vaddr. Permissions (read, write, execute) are set based on segment flags.
  3. If dynamically linked, the kernel identifies the dynamic linker (interpreter, usually specified in a PT*INTERP segment) and loads/runs it. The dynamic linker then loads required shared libraries (identified in PT*DYNAMIC segment) into memory and resolves symbols.
  4. Kernel sets up the initial stack for the process.
  5. Kernel transfers control to the program's entry point (or dynamic linker's entry point).

- **Static vs. Dynamic Linking**:

  - **Static Linking**: All library routines used by the program are copied into the final executable image by the linker at compile time.

Sold to
cxl0603@gmail.com

        - Advantages: Executable is self-contained, no external library dependencies at runtime, potentially faster startup.
        - Disadvantages: Larger executable file size, if library has a bug/vulnerability, every statically linked program needs to be recompiled.

    - **Dynamic Linking**: The names of shared libraries are placed in the executable. Actual linking with library routines occurs at runtime when the program is loaded (by the dynamic linker) or when a library function is first called.

        - Advantages: Smaller executable size, shared libraries are loaded into memory once and shared by multiple processes (saves RAM), libraries can be updated independently of programs.
        - Disadvantages: Requires shared libraries to be present on the system, potential for version conflicts ("DLL hell") if dependencies are not managed correctly.

- **Linker (Link Editor)**: A program that takes one or more object files (produced by compilers/assemblers) and combines them into a single executable, library file, or another object file. Resolves symbol references between object files.
- **Loader**: Part of the OS responsible for loading programs and libraries from secondary storage into main memory and preparing them for execution.

Ctrl+C (SIGINT)

1. User presses Ctrl+C on the keyboard.
2. Keyboard hardware generates an electrical signal. The keyboard controller sends a scan code to the computer.
3. The keyboard device driver (kernel module) receives the scan code and interprets it as Ctrl+C.
4. The TTY (teletype) driver (or terminal emulator) in the kernel, associated with the foreground process group, recognizes Ctrl+C as the interrupt character (`VINTR`).
5. The TTY driver sends a `SIGINT` (Signal Interrupt, typically signal 2) to all processes in the foreground process group of the current terminal.
6. **Kernel Signal Handling**:

    - The `SIGINT` signal is delivered to the target process(es). The signal is marked pending in the process's PCB.
    - If the process is in kernel mode (e.g., in a system call), handling is usually deferred until it returns to user mode.
    - When the process is scheduled to run (or is about to return to user mode), the kernel checks for pending signals.
    - **Signal Disposition**:

Sold to
cxl0603@gmail.com

- If the process has a custom signal handler for `SIGINT`, that handler is invoked.
- If the process is ignoring `SIGINT` (`SIG_IGN`), nothing happens.
- If the process has the default action for `SIGINT` (`SIG_DFL`), the default action is to terminate the process.

7. The process terminates (if default action or handler exits) or continues (if handler handles it and doesn't exit, or if ignored). The hardware interrupt itself (from keyboard) would have been handled quickly by its ISR, causing a mode switch from user to kernel, then back to user (or the signal handling path).

Sold to
cxl0603@gmail.com